1. Auditable N-Gram External Memory for Small Language Models: Capabilities and Boundaries

Authors: qwen35-ple project

Date: 2026-09-06

1.1. Abstract

External memory is often proposed as a way to improve small language models without expensive retraining. This paper studies a specific instance: an auditable, n-gram-addressable external memory built from a Qwen3.8-Flash-Next PLE table and attached to a frozen Qwen3.5-0.8B model. We introduce **PLE Projector**, a small MLP that maps the backbone hidden state and lexical memory features to per-token scale/bias corrections in logit space, and a token-level learned policy that protects open-ended generation.

On a real HumanEval subset, BM25+PLE recovers problem-level passes that the base model does not solve. On 10k local-continuation data with three seeds, the learned projector improves teacher-forced NLL by 0.26 over fixed PLE calibration with bootstrap 95% CI $[0.12, 0.42]$. Official NGM and a small kNN-LM baseline did not match these gains. However, on TriviaQA, the 0.8B base model achieves zero exact match, and raw projector fusion can degrade open-ended generation. The paper therefore positions PLE as a **local low-entropy memory** rather than a universal semantic memory.

1.2. Introduction

Large language models increasingly rely on retrieval and external memory to compensate for limited parameters. For a 0.8B model, parameter-efficient adapters and RAG are practical, but it remains unclear whether an exact n-gram memory can provide a distinct, auditable benefit. DeepSeek's Engram and Qwen's PLE suggest that n-gram lookup can be integrated into a frozen backbone, but the value of such memory on small models is not well characterized.

We study the following questions:

- Can an auditable n-gram external memory improve local code continuation?
- Can a learned projector outperform fixed calibration?
- Can a token-level policy prevent open-ended generation degradation?
- Where does external memory fail?

Our contributions are:

1. A **PLE Projector**: a small MLP that maps hidden states and memory statistics to logit scale/bias, trained with frozen backbone and next-token cross-entropy.
2. A token-level learned policy for safe activation of PLE fusion.
3. Real-benchmark evidence, including HumanEval, TriviaQA, NGM and kNN-LM baselines.
4. A systematic boundary analysis: external n-gram memory helps local low-entropy code, but does not substitute for RAG or parametric adapters on general tasks.

1.3. Related Work

External memory. DeepSeek Engram (link: https://github.com/deepseek-ai/Engram) and Qwen PLE use n-gram lookup with gating and residual injection. Memory Grafting (link: https://papers.cool/arxiv/2605.20948) scales pre-training via offline conditional memory. XMemTransfer (link: https://github.com/OLAResearch/XMemTransfer) transfers memory across models with a target-side reader.

Nonparametric baselines. kNN-LM (link: https://papers.lunadong.com/paper/4555) is a classic nonparametric next-token model, but is known not to improve open-ended generation (link: https://aclanthology.org/2023.emnlp-main.929/). NGM (link: https://github.com/PioneerQyw/NGM) provides a training-free n-gram memory hook. We compare against both.

Parameter-efficient adapters. MoRA, LoRA, and QLoRA provide parametric capability gains. In this paper, these are used as system components rather than replacing external memory.

1.4. Method

1.4.1. PLE Addressable Memory

We use an n-gram-addressable memory:

```
context n-gram -> empirical next-token distribution
             -> external value index
```

For a context c, the memory returns a sparse distribution p_m over the vocabulary and the longest matched n-gram order. This is a fully auditable, non-parametric memory.

1.4.2. PLE Projector

The projector is a small MLP f_θ:

```
h_t (frozen backbone hidden state)
+ memory features (order, entropies, density ratio, agreement)
+ task one-hot (code/name/number/general)
  -> scale_t, bias_t
  -> fused = base_logits + scale_t * log p_m + bias_t
```

The final linear head is zero-initialized, so an untrained projector is identical to the base model. Only the projector is trained; the backbone remains frozen.

1.4.3. Token-Level Learned Policy

A logistic regressor predicts whether PLE fusion will improve the next-token probability, using the same memory features. It acts as a safety gate before PLE fusion, especially for open-ended generation.

1.4.4. Purified OPSD Adapter

We also train a Purified OPSD MoRA adapter on a filtered subset of synthetic instruction data. This provides a parametric companion to the non-parametric PLE memory.

1.5. Experimental Setup

Model. Qwen3.5-0.8B, frozen when used with PLE.

Memory. Qwen3.8-Flash-Next PLE / addressable n-gram table, built from a local Python code corpus and wiki text.

Datasets.

1. HumanEval: official `openai/openai_humaneval`, 20 problems.
2. TriviaQA: official `mandarjoshi/trivia_qa` RC, 100 validation examples.
3. Local continuation: projector datasets of 1k and 10k samples.
4. Joint system tasks: knowledge, arithmetic, code-output subsets.

Baselines.

1. Base frozen model.
2. BM25 RAG.
3. kNN-LM (small hidden-state version).
4. NGM official hook.
5. LoRA / QLoRA / MoRA / Purified MoRA.
6. Fixed PLE calibration and learned PLE Projector.

1.6. Results

1.6.1. HumanEval

Condition	pass@1	mean repetition
Base	0.10	0.010
BM25+PLE	0.10	0.026

Table 1: HumanEval 20: pass@1 and repetition.

Base solved `HumanEval/16` and `HumanEval/18`. BM25+PLE solved `HumanEval/0` and `HumanEval/10`. The solved sets are disjoint, indicating that PLE retrieval provides different local code knowledge rather than duplicating the base model's ability.

1.6.2. TriviaQA

For 100 TriviaQA RC examples, the 0.8B base model obtained:

```
exact match = 0.0
mean repetition = 0.0048
```

This demonstrates that the model cannot yet answer short-form knowledge questions reliably, and that PLE does not magically fix this.

1.6.3. Local Continuation: 10k Data, 3 Seeds

Seed	Base NLL	Fixed NLL	Projector NLL	Base hit	Fixed hit	Projector hit
0	3.148	3.051	2.930	0.407	0.427	0.463
1	3.328	3.346	3.114	0.410	0.420	0.460
2	3.451	3.426	3.002	0.413	0.430	0.493

Table 2: 10k local continuation: projectors vs fixed calibration.

Paired across seeds:

1. Projector vs fixed NLL mean: $+0.2595$
2. Bootstrap 95% CI: $[0.1218, 0.4243]$
3. Positive seeds: $\frac{3}{3}$

At 100 samples with 5 seeds, the projector-vs-fixed mean was $+0.1044$ with CI $[0.0251, 0.1866]$, also positive.

1.6.4. NGM and kNN-LM Baselines

Baseline	NLL	Hit	Note
Base	2.521	0.550	
NGM	2.521	0.550	near-zero change
Base	2.633	0.505	kNN eval set
kNN-LM	2.847	0.540	better hit, worse NLL

Table 3: Official NGM and small-scale kNN-LM baselines.

Neither contemporary training-free baseline outperformed the learned PLE projector on local continuation.

1.6.5. Joint System Table

On knowledge, arithmetic, and code-output tasks with 3 seeds:

System	Knowledge	Arithmetic	Code-output
Base	−8.140	−7.365	−14.250
RAG	−6.748	−7.365	−14.250
PLE	−8.140	−7.492	−14.250
MoRA	−7.691	−7.114	−12.292
RAG+MoRA	−6.704	−7.114	−12.292
All	−6.704	−7.237	−12.292

Table 4: Mean answer log-probability (higher is better).

RAG mainly improves knowledge; MoRA mainly improves arithmetic and code-output; PLE alone does not improve general tasks.

1.6.6. Open-Ended Generation Safety

Raw PLE projector fusion on five natural-language code prompts produced visible degradation: repetitive fragments, unrelated repository text, and broken structure. Adding the learned token policy strongly reduced this degradation. This indicates that PLE should not be enabled unconditionally in open-ended generation.

1.7. Discussion

The empirical picture is clear:

1. PLE n-gram memory is most valuable as a **local, low-entropy code memory**.
2. A learned projector is better than fixed calibration when enough local data is available.
3. RAG and parametric adapters remain superior for general knowledge and reasoning.
4. PLE is not a universal semantic memory, and overselling it would be wrong.

1.8. Limitations

1. HumanEval subset is 20 problems; TriviaQA exact match is zero.
2. Pass@k evidence is small (3 problems x 2 samples).
3. LLM-as-judge results are not yet available in the published artifact package.
4. Public adapter weights are not yet released.
5. CPU deployment throughput is not yet optimized.

1.9. Conclusion

We presented an auditable n-gram external memory system for a 0.8B frozen model, including a learned PLE projector, token-level policy, real-benchmark evaluations, and comparisons against contemporary external-memory baselines. The results support a boundary claim: external n-gram memory can improve local low-entropy code continuation, while RAG and parametric adapters remain necessary for general ability. The system is fully reproducible with open source code, data builders, evaluation scripts, and a container.

1.10. Reproducibility

All code is available in the repository. Key artifacts include:

1. `scripts/run_humaneval_real_ablation.py`
2. `scripts/run_humaneval_passk.py`
3. `scripts/run_triviaqa_real_eval.py`
4. `scripts/run_ngm_baseline.py`
5. `scripts/run_knn_lm_baseline.py`
6. `scripts/run_10k_projector_seeds.sh`
7. `Dockerfile`
8. `docs/evaluation-card-paper.md`

1.11. References

1. DeepSeek Engram: https://github.com/deepseek-ai/Engram
2. Memory Grafting: https://papers.cool/arxiv/2605.20948
3. XMemTransfer: https://github.com/OLAResearch/XMemTransfer
4. NGM: https://github.com/PioneerQyw/NGM
5. kNN-LM: https://papers.lunadong.com/paper/4555
6. kNN-LM Does Not Improve Open-ended Text Generation: https://aclanthology.org/2023.emnlp-main.929/
7. Memory Layers at Scale: https://mlanthology.org/icml/2025/berges2025icml-memory/
8. MemSFT: https://github.com/LUMIA-Group/MemSFT